llm-quantization-benchmark: side-by-side comparison of LLM quantization recipes

Akshitha Reddy Lingampally

2026-06-06

Contents

1 Abstract

We present `llm-quantization-benchmark`, a single-command harness for comparing LLM quantization recipes (fp32, fp16, bnb 8-bit, bnb 4-bit FP4 + NF4, GPTQ 4-bit, AWQ 4-bit) on the

same model and reporting perplexity, model size on device, peak GPU memory, load time, and per-token decode latency in one consolidated table. The point is to turn "should I quantize?" from a vibe into a real cost-quality plot for *your* model on *your* eval text. We report the fp16 baseline on Qwen2.5-0.5B-Instruct (CPU-only smoke run): PPL = 16.3 on wikitext-2, 942 MB model size, 1.7s load, 12.8ms/token decode. The bnb recipes require a CUDA GPU and are skipped cleanly on CPU; running on a GPU box exercises the full sweep.

2 1. Background

LLM quantization is the standard production lever for trading a small amount of quality for a large amount of memory and latency. The recipes split into two families: post-training quantization (PTQ — GPTQ, AWQ, bitsandbytes) and quantization-aware training (QAT — out of scope here). Within PTQ, the choices are:

- **fp16/fp32**: no quantization; baseline.
- **bnb 8-bit** (Dettmers et al., 2022): per-row LLM.int8() mixed- precision matmul.
- **bnb 4-bit FP4 / NF4** (Dettmers et al., 2023, QLoRA): 4-bit float / normal-float types with paged block decompression.
- **GPTQ 4-bit** (Frantar et al., 2023): per-block Hessian-based quantization with a calibration set.
- **AWQ 4-bit** (Lin et al., 2023): activation-aware quantization; preserves outlier weight rows.

Most published quantization comparisons report a single model on wikitext perplexity; this harness makes it cheap to repeat that comparison on the user's model + corpus.

3 2. Related Work

- **GPTQ** (Frantar et al., 2023): the gold-standard 4-bit PTQ method for LLMs.
- **AWQ** (Lin et al., 2023): newer competitor to GPTQ; per-channel activation-aware scaling.
- **bitsandbytes / LLM.int8()** (Dettmers et al., 2022): the production-default 8-bit and 4-bit quantization, integrated into transformers.
- **QLoRA** (Dettmers et al., 2023): NF4 + paged optimizer for fine-tuning quantized models.

4 3. Method

4.1 3.1 Recipe registry

recipe	bits	method	needs
fp32	32	none	torch
fp16	16	none	torch
bnb_8bit	8	bitsandbytes	bitsandbytes + CUDA

recipe	bits	method	needs
bnb_4bit	4	bitsandbytes	bitsandbytes + CUDA (FP4)
bnb_nf4	4	bitsandbytes	bitsandbytes + CUDA (NF4)
gptq_4bit	4	gptq	pre-quantized GPTQ weights
awq_4bit	4	awq	pre-quantized AWQ weights

The bnb recipes need CUDA at load time; CPU-only machines will see them fail to load and the harness skips them. GPTQ and AWQ expect a pre-quantized variant of the model on the Hub (e.g. `TheBloke/Qwen2.5-0.5B-GPTQ`).

4.2 3.2 Perplexity

Sliding-window perplexity on wikitext-2 with `max_length=512` and `stride=256`. Matches the HuggingFace evaluate-perplexity recipe.

4.3 3.3 Latency

Warmup + timed `generate(max_new_tokens=64, do_sample=False)` divided by new tokens. Wall-clock, single-threaded.

4.4 3.4 Size accounting

Model size on device = sum(params) × bits / 8. Holds for dense layers; quantized weights with packing report through `.numel()` but actual storage is bits/8 of that count.

5 4. Data

- **Perplexity corpus**: wikitext-2 test split, first 50K characters (HF: `wikitext`, config `wikitext-2-raw-v1`).
- **Latency prompt**: “The capital of France is” (fixed, single prompt, 64 new tokens).

6 5. Evaluation Setup

Hardware for the published run: Apple M-series, CPU only. Recipes run: fp16 only (bnb recipes need CUDA; GPTQ/AWQ need pre-quantized variants for the chosen model). Adding a GPU and the full recipe sweep is a one-flag CLI change.

7 6. Results

recipe	bits	PPL	size MB	peak GPU MB	load s	ms/tok
fp16	16	16.3	942.3	0.0	1.7	12.8

Three honest observations from this baseline:

1. **PPL = 16.3 is consistent with a small 0.5B model.** Published Qwen2.5-0.5B numbers are around 14-15; the 1-2 point gap is the sliding-window vs full-sequence eval difference plus the 50k-char sample size cap. Good enough to compare *between* recipes on the same harness; do not compare across leaderboards.
2. **Peak GPU memory = 0** because the run was CPU. The same column on a GPU run is the headline number for the bnb recipes (they cut it by roughly half going from fp16 to 8-bit, and roughly to one quarter going to 4-bit).
3. **Decode latency = 12.8 ms/token** on CPU for a 0.5B model. The same number on a small GPU is well under 1 ms.

The harness's value is in the comparison. With a GPU box, a single `uv run qs bench --model ... --recipes fp16,bnb_8bit,bnb_4bit,bnb_nf4` produces the headline cost-quality plot.

8 7. Ablations

Pending; the obvious next sweep is per-recipe with multiple model sizes (0.5B, 1.5B, 3B, 7B) to confirm that quantization sensitivity scales as expected with parameter count.

9 8. Discussion

For deployment, the question is rarely "what's the lowest perplexity" but "what's the lowest cost at acceptable perplexity." The harness's Pareto plot answers exactly that. The conventional wisdom is:

- bnb 8-bit is essentially free quality-wise; if you have CUDA, use it.
- bnb 4-bit NF4 loses 0-2% quality on standard benchmarks for ~75% memory reduction; the safe 4-bit default.
- GPTQ and AWQ can hit better quality at 4-bit but need calibration.

These numbers are hard claims about the production setup; the harness lets you verify them on your specific model.

10 9. Limitations

1. **CPU-only smoke run.** Only fp16 in the reported numbers; full sweep needs a CUDA box.

2. **Wikitext-2 only.** Domain-specific models will see different quantization sensitivity on domain text.
3. **Single hardware target.** Latency numbers don't carry across GPU types.
4. **No QAT.** Only post-training quantization is in scope.

10. Future Work

- ☐ GPU sweep across all 7 recipes on Qwen2.5-1.5B, 3B, 7B.
- ☐ In-process GPTQ calibration via AutoGPTQ (skip the pre-quantized-weight requirement).
- ☐ In-process AWQ calibration via AutoAWQ.
- ☐ Downstream-task accuracy (MMLU, GSM8K) alongside perplexity.
- ☐ Apple Silicon MLX path for the 4-bit recipes.

11. References

- Dettmers, T., et al. (2022). *LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale.* NeurIPS. arXiv:2208.07339.
- Dettmers, T., et al. (2023). *QLoRA: Efficient Finetuning of Quantized LLMs.* NeurIPS. arXiv:2305.14314.
- Frantar, E., et al. (2023). *GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers.* ICLR. arXiv:2210.17323.
- Lin, J., et al. (2024). *AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration.* MLSys. arXiv:2306.00978.
- Yang, A., et al. (2024). *Qwen2.5 Technical Report.* arXiv:2412.15115.

Appendix A. Reproducibility

- Repo: `Akshitha024/llm-quantization-benchmark`, MIT.
- Reproduce fp16 baseline: `uv run qs bench --model Qwen/Qwen2.5-0.5B-Instruct --recipes fp16`.
- Full sweep: `uv run qs bench --model <m> --recipes fp16,bnb_8bit,bnb_4bit,bnb_nf4,gptq_4`
- Test artifacts in `docs/test_results/`.